

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

OPERATING SYSTEMS (23CS4PCOPS)

Submitted by

**C.SMITHA
(1BM24CS076)**

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**OPERATING SYSTEMS**” carried out by **C.SMITHA (1BM24CS076)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of **OPERATING SYSTEMS - (23CS4PCOPS)** work prescribed for the said degree.

Ramya K M
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF c) Priority d) Round Robin	5-20
2	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	21-25
3	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	26-39
4	Write a C program to simulate: a) Producer-Consumer problem using semaphores b) Dining-Philosopher's problem	40-46
5	Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance b) Deadlock Detection	47-56
6	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	57-62
7	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	63-69

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

a) First Come First Serve (FCFS)

```
#include<stdio.h>

int main()
{
    int n,i,j;
    printf("enter the number of process");
    scanf("%d",&n);
    int AT[n],BT[n],p[n];
    for (i=0;i<n;i++)
    {
        printf("enter the arrival time of the process %d",i+1);
        scanf("%d",&AT[i]);
        printf("enter the Burst time of the process %d",i+1);
        scanf("%d",&BT[i]);
        p[i]=i+1;
    }
    for(i=0;i<n-1;i++)
    {
        for(j=0;j<n-i-1;j++)
        {
            if(AT[j] > AT[j+1])
            {
                int temp = AT[j];
                AT[j] = AT[j+1];
                AT[j+1] = temp;

                temp = BT[j];
```

```

        BT[j] = BT[j+1];
        BT[j+1] = temp;
        temp = p[j];
        p[j] = p[j+1];
        p[j+1] = temp;
    }
}
}
int current_time=0;
int TAT[n],CT[n],WT[n];
float sumWT=0;
float sumTAT=0;
for(i=0;i<n;i++)
{
    if(current_time<AT[i])
        current_time=AT[i];

    CT[i]=current_time+BT[i];
    TAT[i]=CT[i]-AT[i];
    WT[i]=TAT[i]-BT[i];

    current_time=CT[i];

    sumWT=sumWT+WT[i];
    sumTAT=sumTAT+TAT[i];

}
float avgWT=sumWT/n;
float avgTAT=sumTAT/n;

```

```

for(i=0;i<n;i++)
{
    printf(" \nThe process id %d",p[i]);
    printf("\nThe Arrival time %d",AT[i]);
    printf("\nThe Burst Time %d",BT[i]);
    printf("\nThe Completion time %d",CT[i]);
    printf("\nThe waiting Time %d",WT[i]);
    printf("\nThe turn around time %d",TAT[i]);
}

printf("\nThe Average waiting time %f",avgWT);
printf("\nThe Average Turnaround time %f",avgTAT);

return 0;
}

```

Output:

The screenshot shows the Code::Blocks IDE with a C program being executed. The console output is as follows:

```

enter the number of process 2
enter the arrival time of the process 1 10
enter the Burst time of the process 1 11
enter the arrival time of the process 2 20
enter the Burst time of the process 2 21

The process id 1
The Arrival time 10
The Burst Time 11
The Completion time 21
The waiting Time 0
The turn around time 11
The process id 2
The Arrival time 20
The Burst Time 21
The Completion time 42
The waiting Time 1
The turn Around time 22
The Average waiting time 0.500000
The Average Turnaround time 16.500000
Process returned 0 (0x0)   execution time : 12.618 s
Press any key to continue.

```

The IDE interface includes a file explorer on the left, a central editor window, and a bottom panel showing logs and errors. The status bar at the bottom indicates the current file is C:\Users\BMSCECE-LA\Desktop\10m24c207\ifc.c, line 32, column 1, position 1155.

Observation:

```

1 // FCFS (First come first serve)
2 #include <stdio.h>
3 int main()
4 {
5     int n, i, j, AT[10], BT[10], p[10];
6     printf("Enter the number of processes: ");
7     scanf("%d", &n);
8     for(i=0; i<n; i++)
9     {
10         printf("Enter the arrival time of the process %d", i+1);
11         scanf("%d", &AT[i]);
12         printf("Enter the burst time of the process %d", i+1);
13         scanf("%d", &BT[i]);
14         p[i] = i+1;
15     }
16     for(i=0; i<n-1; i++)
17     {
18         for(j=0; j<n-1-i; j++)
19         {
20             if(AT[j] > AT[j+1])
21             {
22                 int temp = AT[j];
23                 AT[j] = AT[j+1];
24                 AT[j+1] = temp;
25                 temp = BT[j];
26                 BT[j] = BT[j+1];
27                 BT[j+1] = temp;
28                 temp = p[j];
29                 p[j] = p[j+1];
30                 p[j+1] = temp;
31             }
32         }
33     }
34 }

```

```

1 int current_time = 0;
2 int AT[10], CT[10], WT[10];
3 float sumWT = 0;
4 float sumAT = 0;
5 for(i=0; i<n; i++)
6 {
7     if(current_time < AT[i])
8     {
9         current_time = AT[i];
10        CT[i] = current_time - AT[i];
11        TAT[i] = CT[i] - AT[i];
12        WT[i] = TAT[i] - BT[i];
13        current_time = CT[i];
14        sumWT = sumWT + WT[i];
15        sumAT = sumAT + TAT[i];
16    }
17    float avgWT = sumWT/n;
18    float avgTAT = sumTAT/n;
19    printf("In FCFS AT AT BT CT TAT WT\n");
20    for(i=0; i<n; i++)
21    {
22        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t",
23            p[i], AT[i], BT[i], CT[i], TAT[i], WT[i]);
24    }
25    printf("In Average waiting time = %.2f", avgWT);
26    printf("In Average Turnaround time = %.2f", avgTAT);
27    return 0;
28 }

```

Output 1:-

Enter the no. of processes: 2

Enter Arrival time of the process 1: 10

Enter Burst time of the process 1: 11

Enter Arrival time of the process 2: 20

Enter Burst time of the process 2: 21

PID	AT	BT	CT	TAT	WT
1	10	11	21	11	0
2	20	21	42	22	1

Average waiting time: 0.50000

Average Turnaround time: 16.5000

Enter the no. of processes: 3

Enter Arrival time of the process 1: 0

Enter Burst time of the process 1: 5

Enter Arrival time of the process 2: 0

Enter Burst time of the process 2: 3

Enter Arrival time of the process 3: 0

Enter Burst time of the process 3: 8

PID	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	0	3	8	8	5
P3	0	8	16	16	8

(b) Shortest Job First (SJF)

```
#include <stdio.h>
```

```
struct Process
```

```
{
```

```
    int pid;
```

```
    int at;
```

```
    int bt;
```

```
    int ct;
```

```
    int tat;
```

```
    int wt;
```

```
    int completed;
```

```
};
```

```
int main()
```

```
{
```

```
    int n, i;
```

```
    struct Process p[20];
```

```
    int current_time = 0, completed = 0;
```

```
    float avgWT = 0, avgTAT = 0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        p[i].pid = i + 1;
```

```
        printf("Enter Arrival Time and Burst Time for P%d: ", p[i].pid);
```

```
        scanf("%d %d", &p[i].at, &p[i].bt);
```

```
        p[i].completed = 0;
```

```
    }
```

```

while(completed < n)
{
    int index = -1;
    int minBT = 9999;

    for(i = 0; i < n; i++)
    {
        if(p[i].at <= current_time && p[i].completed == 0)
        {
            if(p[i].bt < minBT)
            {
                minBT = p[i].bt;
                index = i;
            }
        }
    }
    if(index == -1)
    {
        current_time++;
    }
    else
    {
        p[index].ct = current_time + p[index].bt;
        p[index].tat = p[index].ct - p[index].at;
        p[index].wt = p[index].tat - p[index].bt;

        current_time = p[index].ct;
        p[index].completed = 1;
        completed++;
    }
}

```

```

        avgWT += p[index].wt;
        avgTAT += p[index].tat;
    }
}
avgWT /= n;
avgTAT /= n;
printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
}

printf("\nAverage Waiting Time = %.2f", avgWT);
printf("\nAverage Turnaround Time = %.2f\n", avgTAT);
return 0;
}

```

Output:

```

C:\Users\BMSCECE-L4\Desktop x + v
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 6
Enter Arrival Time and Burst Time for P2: 2 8
Enter Arrival Time and Burst Time for P3: 4 3

PID   AT   BT   CT   TAT  WT
P1     0    6    6    6    0
P2     2    8   17   15    7
P3     4    3    9    5    2

Average Waiting Time = 3.00
Average Turnaround Time = 8.67

Process returned 0 (0x0)   execution time : 10.646 s
Press any key to continue.
|

```

Observation:

```

a2 STF
#include <stdio.h>
#define Process 3
int pId;
int AT;
int BT;
int CT;
int TAT;
int WT;
int completed;
int n;
int main()
{
    struct Process p[10];
    int current_time = 0, completed = 0;
    float avgWT = 0, avgTAT = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for(i=0; i<n; i++)
    {
        p[i].pId = i+1;
        printf("Enter Arrival Time and Burst time for p%d: ", p[i].pId);
        scanf("%d %d", &p[i].at, &p[i].bt);
        p[i].completed = 0;
    }
    while(completed < n)
    {
        int index = -1;
        int minBT = 9999;
    }
}

```

```

    for(i=0; i<n; i++)
    {
        if(p[i].AT <= current_time && p[i].completed == 0)
        {
            if(p[i].BT < minBT)
            {
                minBT = p[i].BT;
                index = i;
            }
        }
        if(index == -1)
        {
            current_time++;
        }
        else
        {
            p[index].CT = current_time + p[index].BT;
            p[index].TAT = p[index].CT - p[index].AT;
            p[index].WT = p[index].TAT - p[index].BT;
            current_time = p[index].CT;
            p[index].completed = 1;
            completed++;
            avgWT += p[index].WT;
            avgTAT += p[index].TAT;
        }
        avgWT /= n;
        avgTAT /= n;
        printf("In PID AT BT CT TAT WT\n");
        for(i=0; i<n; i++)
        {
            printf("P%d\t%d\t%d\t%d\t%d\t%d\n", p[i].pId,
                p[i].AT, p[i].BT, p[i].CT, p[i].TAT, p[i].WT);
        }
        printf("Average waiting time = %.2f", avgWT);
        printf("Average Turnaround time = %.2f", avgTAT);
        return 0;
    }
}

```

Output:-

Enter number of processes: 3
 Enter Arrival Time and Burst time for p1: 0 6
 Enter Arrival Time and Burst time for p2: 2 8
 Enter Arrival Time and Burst time for p3: 4 3

PID	AT	BT	CT	TAT	WT
p1	0	6	6	6	0
p2	2	8	17	15	7
p3	4	3	9	5	2

Average waiting Time = 3.00
 Average Turnaround time = 8.67

Run

P1	P3	P2
0	6	9
6	9	17

(c) Priority Scheduling

```
#include <stdio.h>

int main() {
    int n, i, completed = 0, current_time = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int AT[n], BT[n], Priority[n];
    int CT[n], TAT[n], WT[n];
    int completed_flag[n];

    for(i = 0; i < n; i++) {
        printf("\nProcess %d:\n", i + 1);
        printf("Arrival Time: ");
        scanf("%d", &AT[i]);
        printf("Burst Time: ");
        scanf("%d", &BT[i]);
        printf("Priority (smaller value = higher priority): ");
        scanf("%d", &Priority[i]);
        completed_flag[i] = 0;
    }

    while(completed < n) {
        int idx = -1;
        int highest_priority = 9999;

        for(i = 0; i < n; i++) {
            if(AT[i] <= current_time && completed_flag[i] == 0) {
                if(Priority[i] < highest_priority) {
```

```

        highest_priority = Priority[i];
        idx = i;
    }

    else if(Priority[i] == highest_priority) {
        if(AT[i] < AT[idx]) {
            idx = i;
        }
    }
}

if(idx != -1) {
    CT[idx] = current_time + BT[idx];
    TAT[idx] = CT[idx] - AT[idx];
    WT[idx] = TAT[idx] - BT[idx];

    current_time = CT[idx];
    completed_flag[idx] = 1;
    completed++;
} else {
    current_time++;
}
}

float total_WT = 0, total_TAT = 0;

printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++) {

```

```

printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
       i + 1, AT[i], BT[i], Priority[i],
       CT[i], TAT[i], WT[i]);

total_WT += WT[i];
total_TAT += TAT[i];
}

printf("\nAverage Waiting Time = %.2f", total_WT / n);
printf("\nAverage Turnaround Time = %.2f\n", total_TAT / n);

return 0;
}

```

Output:

```

Process 1:
Arrival Time: 0
Burst Time: 10
Priority (smaller value = higher priority): 2

Process 2:
Arrival Time: 1
Burst Time: 15
Priority (smaller value = higher priority): 1

PID   AT    BT    PR    CT    TAT   WT
P1     0     10     2     10     10     0
P2     1     15     1     25     24     9

Average Waiting Time = 4.50
Average Turnaround Time = 17.00

Process returned 0 (0x0)   execution time : 13.162 s
Press any key to continue.

```

Observation:

```

3 // priority
#include <iostream>
int main() {
    int n, i, completed = 0, current_time = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int AT[n], BT[n], Priority[n];
    int CT[n], TAT[n], WT[n];
    int completed_flag[n];
    for (i = 0; i < n; i++) {
        printf("Process %d: ", i+1);
        printf("Arrival Time: ");
        scanf("%d", &AT[i]);
        printf("Burst Time: ");
        scanf("%d", &BT[i]);
        printf("Priority: ");
        scanf("%d", &Priority[i]);
        completed_flag[i] = 0;
    }
    while (completed < n) {
        int idx = -1;
        for (i = 0; i < n; i++) {
            if (AT[i] < current_time && completed_flag[i] == 0) {
                if (Priority[i] < high_p) {
                    high_p = Priority[i];
                    idx = i;
                }
            }
        }
        if (idx != -1) {
            current_time += BT[idx];
            completed_flag[idx] = 1;
            completed++;
            CT[idx] = current_time - AT[idx];
            TAT[idx] = CT[idx] - BT[idx];
            WT[idx] = TAT[idx] - BT[idx];
        }
    }
    printf("Total TAT = ");
    for (i = 0; i < n; i++) {
        TAT[i] = CT[i] - BT[i];
    }
    printf("Total WT = ");
    for (i = 0; i < n; i++) {
        WT[i] = TAT[i] - BT[i];
    }
    printf("Average TAT = ");
    for (i = 0; i < n; i++) {
        TAT[i] = CT[i] - BT[i];
    }
    printf("Average WT = ");
    for (i = 0; i < n; i++) {
        WT[i] = TAT[i] - BT[i];
    }
    return 0;
}

```

```

else if (Priority[i] == high_p) {
    if (AT[i] < AT[idx]) {
        idx = i;
    }
}
if (idx != -1) {
    CT[idx] = current_time + BT[idx];
    TAT[idx] = CT[idx] - AT[idx];
    WT[idx] = TAT[idx] - BT[idx];
    current_time = CT[idx];
    completed_flag[idx] = 1;
    completed++;
} else {
    current_time++;
}
// Total TAT = 0, total WT = 0;
printf("In PTD it AT is PR is CT is TAT is WT is\n");
for (i = 0; i < n; i++) {
    printf("P%d: AT %d PR %d CT %d TAT %d WT %d\n",
           i+1, AT[i], BT[i], Priority[i], CT[i], TAT[i],
           WT[i]);
    total_WT += WT[i];
    total_TAT += TAT[i];
}
printf("Average waiting Time = %.2f", total_WT/n);
printf("Average TAT = %.2f", total_TAT/n);
return 0;
}

```

O/P:-

Enter number of process: 2

Process 1:
 Arrival Time: 0
 Burst Time: 10
 Priority: 2

Process 2:
 Arrival time: 1
 Burst time: 15
 Priority: 1

PID	AT	BT	PR	CT	TAT	WT
P1	0	10	2	10	10	0
P2	1	15	1	25	24	9

Average waiting Time: 4.50
 Average TAT: 17.00

(d) Round Robin Scheduling

```
#include <stdio.h>

#define MAX 100

int main() {
    int n, TQ;
    int AT[MAX], BT[MAX], RT[MAX];
    int CT[MAX], WT[MAX], TAT[MAX];
    int queue[MAX], front = 0, rear = 0;
    int visited[MAX] = {0};
    int time = 0, completed = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for P%d: ", i);
        scanf("%d %d", &AT[i], &BT[i]);
        RT[i] = BT[i];
    }
    printf("Enter Time Quantum: ");
    scanf("%d", &TQ);
    for (int i = 0; i < n; i++) {
        if (AT[i] <= time) {
            queue[rear++] = i;
            visited[i] = 1;
        }
    }
    while (completed < n) {
        if (front == rear) {
            time++;
            for (int i = 0; i < n; i++) {
                if (!visited[i] && AT[i] <= time) {
```

```

        queue[rear++] = i;
        visited[i] = 1;
    }
}
continue;
}
int p = queue[front++];
if (RT[p] > TQ) {
    time += TQ;
    RT[p] -= TQ;
} else {
    time += RT[p];
    RT[p] = 0;
    CT[p] = time;
    completed++;
}
for (int i = 0; i < n; i++) {
    if (!visited[i] && AT[i] <= time) {
        queue[rear++] = i;
        visited[i] = 1;
    }
}
if (RT[p] > 0) {
    queue[rear++] = p;
}
}
float totalTAT = 0, totalWT = 0;
for (int i = 0; i < n; i++) {
    TAT[i] = CT[i] - AT[i];
    WT[i] = TAT[i] - BT[i];
}

```

```

        totalTAT += TAT[i];

        totalWT += WT[i];
    }
    printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            i, AT[i], BT[i], CT[i], TAT[i], WT[i]);
    }
    printf("\nAverage Turnaround Time = %.2f", totalTAT / n);
    printf("\nAverage Waiting Time = %.2f\n", totalWT / n);

    return 0;
}

```

Output:

```

C:\Users\BMSCECSE-L4\Docu... X + v
Enter number of processes: 5
Enter Arrival Time and Burst Time for P0: 0 8
Enter Arrival Time and Burst Time for P1: 5 2
Enter Arrival Time and Burst Time for P2: 1 7
Enter Arrival Time and Burst Time for P3: 6 3
Enter Arrival Time and Burst Time for P4: 8 5
Enter Time Quantum: 3

P    AT    BT    CT    TAT    WT
P0   0     8     22    22     14
P1   5     2     11     6      4
P2   1     7     23    22     15
P3   6     3     14     8      5
P4   8     5     25    17     12

Average Turnaround Time = 15.00
Average Waiting Time = 10.00

Process returned 0 (0x0)   execution time : 43.695 s
Press any key to continue.

```

Observation:

```

4. Round Robin
#include <iostream>
using namespace std;
#define max 100

int main()
{
    int n, tq;
    int a[100], bt[100], rt[100];
    int ct[100], wt[100], tat[100];
    int queue[100], front = 0, rear = 0;
    int visited[100] = {0};

    int time = 0, completed = 0;
    printf("Enter number of process: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
    {
        printf("Enter Arrival Time and Burst Time for P%d", i);
        scanf("%d %d", &a[i], &bt[i]);
        rt[i] = bt[i];
    }

    printf("Enter Quantum Time: ");
    scanf("%d", &tq);
    for (int i = 0; i < n; i++)
    {
        if (a[i] <= time)
        {
            queue[rear++] = i;
            visited[i] = 1;
        }
    }

    while (completed < n)
    {
        if (front == rear)
        {
            time++;
            for (int i = 0; i < n; i++)
            {
                if (!visited[i] && a[i] <= time)
            }
        }
    }

```

```

        queue[rear++] = i;
        visited[i] = 1;
    }
    continue;
}
int p = queue[front++];
if (rt[p] > tq)
{
    time += tq;
    rt[p] -= tq;
}
else
{
    time += rt[p];
    rt[p] = 0;
    ct[p] = time;
    completed++;
}
for (int i = 0; i < n; i++)
{
    if (!visited[i] && a[i] <= time)
    {
        queue[rear++] = i;
        visited[i] = 1;
    }
}

if (rt[p] > 0)
{
    queue[rear++] = p;
    visited[p] = 1;
}

if (rt[p] != 0)
{
    totaltat += 0; totalwt += 0;
    for (int i = 0; i < n; i++)
    {
        tat[i] = ct[i] - a[i];
        wt[i] = tat[i] - bt[i];
        totaltat += tat[i];
        totalwt += wt[i];
    }
}

```

```

printf("In P: tat, bt, rt, ct, wt\n");
for (int i = 0; i < n; i++)
{
    printf("P%d: %d, %d, %d, %d, %d\n", i, a[i], bt[i], ct[i], tat[i], wt[i]);
}

printf("In Average Turnaround time = %.2f", totaltat/n);
printf("In Average waiting time = %.2f", totalwt/n);
return 0;
}

O/p:-
Enter number of processes: 5
Enter Arrival Time and Burst Time for P0: 0 3
Enter Arrival Time and Burst Time for P1: 5 2
Enter Arrival Time and Burst Time for P2: 1 7
Enter Arrival Time and Burst Time for P3: 6 3
Enter Arrival Time and Burst Time for P4: 8 5
Enter Time Quantum: 3

P   AT   BT   CT   TAT   WT
P0   0   3   3   3   0
P1   5   2   11  6   4
P2   1   7   23  22  15
P3   6   3   19  13  7
P4   8   5   25  17  9

Average Turnaround Time = 15.00
Average Waiting Time = 10.00

```

Lab Program 2:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```
#include <stdio.h>

struct process {
    int id, at, bt, type;
    int ct, wt, tat;
};

void sort(struct process p[], int n) {
    struct process temp;
    for(int i = 0; i < n-1; i++) {
        for(int j = i+1; j < n; j++) {
            if(p[i].type > p[j].type ||
               (p[i].type == p[j].type && p[i].at > p[j].at)) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct process p[n];
    int gantt_p[100], gantt_t[100], g = 0;
    for(int i = 0; i < n; i++) {
        p[i].id = i;
```

```

printf("\nProcess %d\n", i);
printf("Enter arrival time: ");
scanf("%d", &p[i].at);
printf("Enter burst time: ");
scanf("%d", &p[i].bt);
printf("Enter type (0 = System, 1 = User): ");
scanf("%d", &p[i].type);
}
int time = 0;
gantt_t[g] = 0;
for(int i = 0; i < n; i++) {
    if(time < p[i].at)
        time = p[i].at;
    gantt_p[g] = p[i].id;
    gantt_t[g] = time;

    time += p[i].bt;

    p[i].ct = time;
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;

    g++;
    gantt_t[g] = time;
}

float total_wt = 0, total_tat = 0;

```

```

printf("\nID\tType\tAT\tBT\tCT\tWT\tTAT\n");

for(int i = 0; i < n; i++) {
    printf("%d\t%s\t%d\t%d\t%d\t%d\t%d\n",
        p[i].id,
        (p[i].type == 0) ? "System" : "User",
        p[i].at, p[i].bt,
        p[i].ct, p[i].wt, p[i].tat);

    total_wt += p[i].wt;
    total_tat += p[i].tat;
}

printf("\nAverage Waiting Time: %.2f", total_wt / n);
printf("\nAverage Turnaround Time: %.2f\n", total_tat / n);

printf("\nGantt Chart:\n");

    printf(" -----");
printf("\n");

for(int i = 0; i < n; i++)
    printf("| P%d  ", gantt_p[i]);
printf("\n");

// Bottom bar
for(int i = 0; i < n; i++)

```

```

        printf(" -----");

printf("\n");

// Time values

for(int i = 0; i <= n; i++)

    printf("%-8d", gantt_t[i]);


printf("\n");


return 0;

}

```

Output:

```

C:\Users\BMSCECSE-L4\Desktop > .\program.exe
Enter number of processes: 3

Process 0
Enter arrival time: 0
Enter burst time: 10
Enter type (0 = System, 1 = User): 0

Process 1
Enter arrival time: 1
Enter burst time: 12
Enter type (0 = System, 1 = User): 1

Process 2
Enter arrival time: 2
Enter burst time: 14
Enter type (0 = System, 1 = User): 0

ID   Type   AT   BT   CT   WT   TAT
0    System 0    10   10    0    10
1     User  1    12   22    9    21
2    System 2    14   36   20    34

Average Waiting Time: 9.67
Average Turnaround Time: 21.67

Gantt Chart:
-----
| P0 | P1 | P2 |
-----
0     10    22    36

Process returned 0 (0x0)   execution time : 18.449 s
Press any key to continue.

```


Observation:

```

10/11/20
Multi-level Queue Scheduling

#include <stdio.h>
struct process {
    int id, at, bt, type;
    int ct, wt, tot;
};

void sort(struct process p[10], int n) {
    struct process temp;
    for (int i = 0; i < n-1; i++) {
        for (int j = i+1; j < n; j++) {
            if (p[i].type > p[j].type ||
                (p[i].type == p[j].type && p[i].at > p[j].at)) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct process p[10];
    int count = 0, gantt = 1, at = 0, g = 0;
    for (int i = 0; i < n; i++) {
        p[i].id = i;
        printf("Enter arrival time: ");
        scanf("%d", &p[i].at);
        printf("Enter burst time: ");
        scanf("%d", &p[i].bt);
    }
}
    
```

```

printf("Enter type (0=System, 1=User): ");
scanf("%d", &p[i].type);
}

int time = 0;
gantt[0] = 0;
for (int i = 0; i < n; i++) {
    if (time < p[i].at) {
        time = p[i].at;
        gantt[p[i].id] = p[i].at;
        gantt[1] = time;
        time = time + p[i].bt;
        p[i].ct = time;
        p[i].wt = p[i].ct - p[i].at;
        p[i].tot = p[i].wt + p[i].bt;
        g++;
        gantt[1] = time;
    }
}

float TWI = 0, TAT = 0;
printf("Enter type (0=System, 1=User): ");
for (int i = 0; i < n; i++) {
    printf("Enter type (0=System, 1=User): ");
    p[i].id, p[i].type == 0 ? "System" : "User";
    p[i].at, p[i].bt, p[i].ct, p[i].wt, p[i].tot;
    TWI += p[i].wt;
    TAT += p[i].tot;
}

printf("Average WT: %.2f", TWI/n);
printf("Average TAT: %.2f", TAT/n);
printf("Gantt Chart: ");
for (int i = 0; i < n; i++) {
    printf("%d ", gantt[p[i].id]);
}
printf("\n");
    
```

```

for (int i = 0; i < n; i++) {
    printf("%d ", gantt[p[i].id]);
}
printf("\n");

for (int i = 0; i < n; i++) {
    printf("Enter type (0=System, 1=User): ");
    p[i].type;
}

// Input
n = 3;
Process 0
Enter arrival time: 0
Enter burst time: 10
Enter type (0=System, 1=User): 0

Process 1
Enter arrival time: 10
Enter burst time: 12
Enter type (0=System, 1=User): 1

Process 2
Enter arrival time: 2
Enter burst time: 14
Enter type (0=System, 1=User): 0

// Output
// Type: AT BT CT WT TAT
// 0 System 0 0 10 0 10
// 1 User 1 10 22 9 21
// 2 System 2 14 26 20 28
    
```

```

Average waiting time: 9.9
Average TAT: 21.67

Gantt chart:
| P0 | P1 | P2 |
0 10 22 26
    
```

Lab Program 3:

Write a C program to simulate Real-Time CPU Scheduling algorithms:

(a) Rate- Monotonic

```
#include <stdio.h>

#include <math.h>

#define MAX 10

int gcd(int a, int b)
{
    while (b != 0)
    {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

int lcm(int a, int b)
{
    return (a * b) / gcd(a, b);
}

int find_lcm(int arr[], int n)
{
    int res = arr[0];

    for (int i = 1; i < n; i++)
    {
        res = lcm(res, arr[i]);
    }
}
```

```

        return res;
    }

int main()
{
    int n;
    int execution[MAX], period[MAX], deadline[MAX];
    int remaining[MAX] = {0};

    float utilization = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
    {
        printf("\nProcess P%d\n", i + 1);

        printf("Enter Execution Time: ");
        scanf("%d", &execution[i]);

        printf("Enter Period: ");
        scanf("%d", &period[i]);

        printf("Enter Deadline: ");
        scanf("%d", &deadline[i]);
    }

    int hyper = find_lcm(period, n);

```

```

printf("\nLCM = %d\n", hyper);

printf("\nRate Monotonic Scheduling\n");

printf("\nProcess\tExecution\tPeriod\tDeadline\n");

for (int i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\n",
        i + 1,
        execution[i],
        period[i],
        deadline[i]);

    utilization += (float)execution[i] / period[i];
}

float bound = n * (pow(2, 1.0 / n) - 1);

printf("\nCPU Utilization = %.2f%%\n", utilization * 100);
printf("RMS Bound = %.2f%%\n", bound * 100);

if (utilization <= bound)
    printf("Scheduling is Possible\n");
else
    printf("Scheduling is Not Possible\n");

printf("\nScheduling occurs for %d ms\n\n", hyper);

int last = -1;

```

```

for (int t = 0; t < hyper; t++)
{
    for (int i = 0; i < n; i++)
    {
        if (t % period[i] == 0)
        {
            remaining[i] = execution[i];
        }
    }
}

int selected = -1;

for (int i = 0; i < n; i++)
{
    if (remaining[i] > 0)
    {
        if (selected == -1 ||
            period[i] < period[selected])
        {
            selected = i;
        }
    }
}

if (selected != last)
{
    if (selected != -1)
        printf("%d ms onwards : Process P%d running\n",
            t, selected + 1);
    else

```

```

        printf("%d ms onwards : CPU Idle\n", t);

        last = selected;
    }
    if (selected != -1)
    {
        remaining[selected]--;
    }
}
return 0;
}

```

Output:

```

C:\Users\BMSCECSE-L4\Desktop X + v
Enter number of processes: 2

Process P1
Enter Execution Time: 20
Enter Period: 50
Enter Deadline: 50

Process P2
Enter Execution Time: 35
Enter Period: 100
Enter Deadline: 100

LCM = 100

Rate Monotonic Scheduling

Process Execution      Period  Deadline
P1      20             50      50
P2      35             100     100

CPU Utilization = 75.00%
RMS Bound = 82.84%
Scheduling is Possible

Scheduling occurs for 100 ms

0 ms onwards : Process P1 running
20 ms onwards : Process P2 running
50 ms onwards : Process P1 running
70 ms onwards : Process P2 running
75 ms onwards : CPU Idle

Process returned 0 (0x0)   execution time : 12.433 s
Press any key to continue.
|

```

Observation:

```

// Rate Monotonic Scheduling
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    int n;
    while (n < 0) {
        cout << "Enter number of processes: ";
        cin >> n;
    }
    vector<int> arr(n);
    vector<int> p(n);
    vector<int> d(n);
    for (int i = 0; i < n; i++) {
        cout << "Enter execution time, period, deadline: ";
        cin >> arr[i] >> p[i] >> d[i];
    }
    sort(arr.begin(), arr.end());
    sort(p.begin(), p.end());
    sort(d.begin(), d.end());
    int i = 0;
    while (i < n) {
        if (arr[i] < p[i] && arr[i] < d[i]) {
            i++;
        } else {
            cout << "Scheduling is not possible\n";
            return 0;
        }
    }
    cout << "Scheduling is possible\n";
    return 0;
}

```

```

// Rate Monotonic Scheduling
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    int n;
    while (n < 0) {
        cout << "Enter number of processes: ";
        cin >> n;
    }
    vector<int> arr(n);
    vector<int> p(n);
    vector<int> d(n);
    for (int i = 0; i < n; i++) {
        cout << "Enter execution time, period, deadline: ";
        cin >> arr[i] >> p[i] >> d[i];
    }
    sort(arr.begin(), arr.end());
    sort(p.begin(), p.end());
    sort(d.begin(), d.end());
    int i = 0;
    while (i < n) {
        if (arr[i] < p[i] && arr[i] < d[i]) {
            i++;
        } else {
            cout << "Scheduling is not possible\n";
            return 0;
        }
    }
    cout << "Scheduling is possible\n";
    return 0;
}

```

```

// Rate Monotonic Scheduling
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    int n;
    while (n < 0) {
        cout << "Enter number of processes: ";
        cin >> n;
    }
    vector<int> arr(n);
    vector<int> p(n);
    vector<int> d(n);
    for (int i = 0; i < n; i++) {
        cout << "Enter execution time, period, deadline: ";
        cin >> arr[i] >> p[i] >> d[i];
    }
    sort(arr.begin(), arr.end());
    sort(p.begin(), p.end());
    sort(d.begin(), d.end());
    int i = 0;
    while (i < n) {
        if (arr[i] < p[i] && arr[i] < d[i]) {
            i++;
        } else {
            cout << "Scheduling is not possible\n";
            return 0;
        }
    }
    cout << "Scheduling is possible\n";
    return 0;
}

```

```

// Rate Monotonic Scheduling
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    int n;
    while (n < 0) {
        cout << "Enter number of processes: ";
        cin >> n;
    }
    vector<int> arr(n);
    vector<int> p(n);
    vector<int> d(n);
    for (int i = 0; i < n; i++) {
        cout << "Enter execution time, period, deadline: ";
        cin >> arr[i] >> p[i] >> d[i];
    }
    sort(arr.begin(), arr.end());
    sort(p.begin(), p.end());
    sort(d.begin(), d.end());
    int i = 0;
    while (i < n) {
        if (arr[i] < p[i] && arr[i] < d[i]) {
            i++;
        } else {
            cout << "Scheduling is not possible\n";
            return 0;
        }
    }
    cout << "Scheduling is possible\n";
    return 0;
}

```

(b) Earliest Deadline First

```
#include <stdio.h>

#define MAX 10

int main()
{
    int n;
    int execution[MAX];
    int period[MAX];
    int deadline[MAX];
    int remaining[MAX];
    int current_deadline[MAX];
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for(int i = 0; i < n; i++)
    {
        printf("\nProcess P%d\n", i + 1);

        printf("Enter Execution Time: ");
        scanf("%d", &execution[i]);

        printf("Enter Period: ");
        scanf("%d", &period[i]);

        printf("Enter Deadline: ");
        scanf("%d", &deadline[i]);

        remaining[i] = execution[i];
        current_deadline[i] = deadline[i];
    }
}
```



```

printf("\nEDF Scheduling Table\n\n");

printf("Time\tReady Queue\t\tRunning\n");

int last = -1;

for(int time = 0; time < 100; time++)
{
    // New process arrival
    for(int i = 0; i < n; i++)
    {
        if(time != 0 && time % period[i] == 0)
        {
            remaining[i] = execution[i];
            current_deadline[i] = time + deadline[i];
        }
    }

    int selected = -1;

    // Select earliest deadline
    for(int i = 0; i < n; i++)
    {
        if(remaining[i] > 0)
        {
            if(selected == -1 ||
               current_deadline[i] < current_deadline[selected])
            {
                selected = i;
            }
        }
    }
}

```

```

    }
}

// Print only when process changes
if(selected != last)
{
    printf("%d\t", time);

    // Ready Queue
    for(int i = 0; i < n; i++)
    {
        if(remaining[i] > 0)
        {
            printf("P%d(%d) ",
                i + 1,
                current_deadline[i]);
        }
    }

    printf("\t\t");

    // Running process
    if(selected != -1)
        printf("P%d\n", selected + 1);
    else
        printf("Idle\n");

    last = selected;
}

```

```

        // Execute selected process
        if(selected != -1)
        {
            remaining[selected]--;
        }
    }
    return 0;
}

```

Output:

```

C:\Users\BMSCECSE-L4\Desktop
Enter number of processes: 2

Process P1
Enter Execution Time: 20
Enter Period: 50
Enter Deadline: 50

Process P2
Enter Execution Time: 35
Enter Period: 80
Enter Deadline: 80

EDF Scheduling Table

Time    Ready Queue    Running
0       P1(50) P2(80)   P1
20      P2(80)         P2
55      P1(100)        P1
75      Idle
80      P2(160)        P2

Process returned 0 (0x0)  execution time : 78.063 s
Press any key to continue.

```

Observation:

```

Earliest Deadline First

#include <stdio.h>
#define MAX 10

int main()
{
    int n;
    int executionTime[];
    int period[];
    int deadline[];
    int remainingTime[];
    int currentDeadline[MAX];
    printf("Enter number of process: ");
    scanf("%d", &n);
    for(i=0; i<n; i++)
    {
        printf("Process P%d\n", i+1);
        printf("Enter Execution Time: ");
        printf("Time at Ready queue (t=0, Running in t): ");
        int last=-1;
        for(int time=0; time<100; time++)
        {
            for(int i=0; i<n; i++)
            {
                if(time%period[i]==0)
                {
                    remainingTime[i]=executionTime[i];
                    currentDeadline[i]=time+deadline[i];
                }
            }
            int select=-1;
            for(int i=0; i<n; i++)
            {
                if(select == -1 || currentDeadline[i] < currentDeadline[select])
                {
                    select = i;
                }
            }
            if(select != -1)
            {
                printf("P%d\n", select+1);
                remainingTime[select]--;
                if(remainingTime[select] == 0)
                {
                    printf("Process P%d completed\n", select+1);
                }
            }
            else
            {
                printf("Idle\n");
            }
            last = select;
        }
    }
}

```

```

if(remainingTime[i] == 0)
{
    if(select == -1 || currentDeadline[i] < currentDeadline[select])
    {
        select = i;
    }
}
if(select == -1)
{
    printf("Idle\n", time);
    for(int i=0; i<n; i++)
    {
        if(remainingTime[i] > 0)
        {
            printf("P%d\n", i+1, currentDeadline[i]);
        }
    }
    printf("t: %d\n", time);
    if(select == -1)
    {
        printf("P%d\n", select+1);
    }
    else
    {
        printf("Idle\n");
        last = select;
    }
}
if(select == -1)
{
    remainingTime[select]--;
}
if(remainingTime[select] == 0)
{
    printf("Process P%d completed\n", select+1);
}
}
}
return 0;
}

```

Q10:

Enter number of process: 2

Process P1

Enter Execution Time: 20

Enter Period: 50

Enter Deadline: 50

Process P2

Enter Execution Time: 25

Enter Period: 80

Enter Deadline: 80

GPF Scheduling Table

Time	Ready Queue	Running
0	P1(50), P2(80)	P1
20	P2(80)	P2
55	P1(100)	P1
75	Idle	Idle
80	P2(160)	P2

(c) Proportional Scheduling

```
#include <stdio.h>

int main()
{
    int n, i, j;
    int totalTime, totalShare = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int share[n], allocation[n];
    printf("Enter total CPU time: ");
    scanf("%d", &totalTime);
    printf("\nEnter share/weight for each process:\n");
    for(i = 0; i < n; i++)
    {
        printf("P%d share: ", i + 1);
        scanf("%d", &share[i]);
        totalShare += share[i];
    }
    for(i = 0; i < n; i++)
    {
        allocation[i] = (share[i] * totalTime) / totalShare;
    }
    printf("\nFinal CPU Allocation:\n");
    for(i = 0; i < n; i++){
        printf("P%d gets %d units of CPU time\n", i + 1, allocation[i]);}
    printf("\nCPU Scheduling Order:\n");
    for(i = 0; i < n; i++)
    {
        printf("P%d", i + 1);
        if(i != n - 1)
```

```

        printf(" -> ");
    }
    printf("\n\nCPU Scheduling Timeline:\n");
    int currentTime = 0;
    for(i = 0; i < n; i++)
    {
        printf("| P%d ", i + 1);
    }
    printf("\n");
    printf("0");
    for(i = 0; i < n; i++)
    {
        currentTime += allocation[i];
        printf("   %d", currentTime);
    }
    return 0;
}

```

Output:

```

C:\Users\BMSCECSE-L4\Desktop X + v
Enter number of processes: 3
Enter total CPU time: 100

Enter share/weight for each process:
P1 share: 2
P2 share: 3
P3 share: 5

Final CPU Allocation:
P1 gets 20 units of CPU time
P2 gets 30 units of CPU time
P3 gets 50 units of CPU time

CPU Scheduling Order:
P1 -> P2 -> P3

CPU Scheduling Timeline:
| P1 | P2 | P3 |
0   20  50   100
Process returned 0 (0x0)   execution time : 8.470 s
Press any key to continue.
|

```

Observation:

```

Proportional CPU Scheduling
#include <stdio.h>
int main()
{
    int n, i, j;
    int totalltime, totalShare=0;
    printf("Enter no. of processes:");
    scanf("%d", &n);
    int share[n], allocation[n];
    printf("Enter total CPU time:");
    scanf("%d", &totalltime);
    printf("Enter share/weight for each process:");
    for(i=0; i<n; i++)
    {
        printf("P%d share: ", i+1);
        scanf("%d", &share[i]);
        totalShare += share[i];
    }
    for(i=0; i<n; i++)
    {
        allocation[i] = (share[i] * totalltime) / totalShare;
    }
    printf("Final CPU Allocation:");
    for(i=0; i<n; i++)
    {
        printf("P%d gets %d units of CPU time\n", i+1, allocation[i]);
    }
    printf("CPU Scheduling Order:");
    for(i=0; i<n; i++)
    {
        printf("P%d", i+1);
        if(i%4 == 3)
            printf("\n");
    }
}

```

```

#include <stdio.h>
int main()
{
    printf("In Proportional CPU Scheduling Time Line:");
    int currenttime = 0;
    for(i=0; i<n; i++)
    {
        printf("P%d", i+1);
        if(i%4 == 3)
            printf("\n");
        currenttime += allocation[i];
        printf(" %d", currenttime);
    }
    return 0;
}

```

Output:-
 Enter no. of processes: 3
 Enter total CPU time: 100
 Enter share/weight for each process:
 P1 share: 2
 P2 share: 3
 P3 share: 5
 Final CPU Allocation
 P1 gets 20 units of CPU time
 P2 gets 30 units of CPU time
 P3 gets 50 units of CPU time
 CPU Scheduling Order: P1 P2 P3
 CPU Scheduling Timeline:
 0 20 50 100

Lab Program 4:

Write a C program to simulate:

(a) Producer-Consumer problem using semaphores

```
#include <stdio.h>

#include <pthread.h>

#include <semaphore.h>

#include <unistd.h>

#define N 8

int buffer[N];

int in = 0, out = 0;

sem_t empty;

sem_t full;

pthread_mutex_t S;

int produce_item() {
    static int item = 1;
    return item++;
}

void consume_item(int item) {
    printf("Consumed: %d\n", item);
}

void* producer(void* arg) {
    int itemP;
    while (1) {
        itemP = produce_item();
        sem_wait(&empty);
        pthread_mutex_lock(&S);
        buffer[in] = itemP;
        printf("Produced: %d at position %d\n", itemP, in);
```



```

        in = (in + 1) % N;
        pthread_mutex_unlock(&S);
        sem_post(&full);
        sleep(1);
    }
}

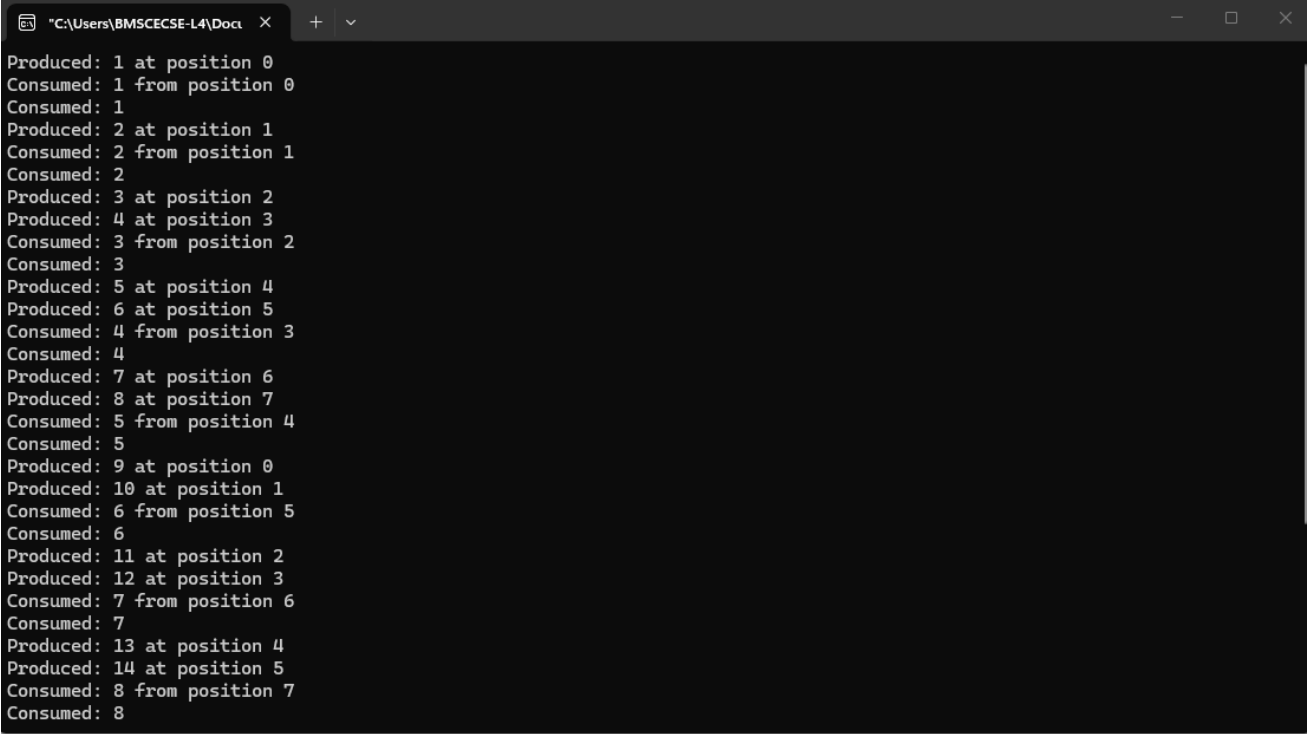
void* consumer(void* arg) {
    int itemC;
    while (1) {
        sem_wait(&full);
        pthread_mutex_lock(&S);
        itemC = buffer[out];
        printf("Consumed: %d from position %d\n", itemC, out);
        out = (out + 1) % N;
        pthread_mutex_unlock(&S);
        sem_post(&empty);
        consume_item(itemC);
        sleep(2);
    }
}

int main() {
    pthread_t prod, cons;
    sem_init(&empty, 0, N);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&S, NULL);
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);
    sem_destroy(&empty);

```

```
sem_destroy(&full);  
pthread_mutex_destroy(&S);  
return 0;  
}
```

Output:



```
"C:\Users\BMSCECSE-L4\Docu x  
Produced: 1 at position 0  
Consumed: 1 from position 0  
Consumed: 1  
Produced: 2 at position 1  
Consumed: 2 from position 1  
Consumed: 2  
Produced: 3 at position 2  
Produced: 4 at position 3  
Consumed: 3 from position 2  
Consumed: 3  
Produced: 5 at position 4  
Produced: 6 at position 5  
Consumed: 4 from position 3  
Consumed: 4  
Produced: 7 at position 6  
Produced: 8 at position 7  
Consumed: 5 from position 4  
Consumed: 5  
Produced: 9 at position 0  
Produced: 10 at position 1  
Consumed: 6 from position 5  
Consumed: 6  
Produced: 11 at position 2  
Produced: 12 at position 3  
Consumed: 7 from position 6  
Consumed: 7  
Produced: 13 at position 4  
Produced: 14 at position 5  
Consumed: 8 from position 7  
Consumed: 8
```

Observation:

```

• Producer-Consumer Problem using Semaphores and pthreads

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define N 8

int buffer[N];
int in=0, out=0;
sem_t empty;
sem_t full;
pthread_mutex_t s;

int produce_item() {
    static int item=1;
    return item++;
}

void consume_item(int item) {
    printf("consumed: %d\n", item);
}

void *Producer(void *arg) {
    int itemp;
    while(1) {
        itemp = produce_item();
        sem_wait(&empty);
        pthread_mutex_lock(&s);
        buffer[in] = itemp;
        printf("Produced: %d at Position %d\n", itemp, in);
        in = (in+1)%N;
        pthread_mutex_unlock(&s);
        sem_post(&full);
    }
}

```

```

    sleep(2);
}

void *consumer(void *arg) {
    int itemc;
    while(1) {
        sem_wait(&full);
        pthread_mutex_lock(&s);
        itemc = buffer[out];
        printf("consumed: %d from Position %d\n", itemc, out);
        out = (out+1)%N;
        pthread_mutex_unlock(&s);
        sem_post(&empty);
        consume_item(itemc);
        sleep(2);
    }
}

int main() {
    pthread_t p, c;
    sem_init(&empty, 0, N);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&s, NULL);
    pthread_create(&p, NULL, Producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&s);
    return 0;
}

```

Date: _____
Page: _____

Op:-

produced: 1 at Position 0

consumed: 1 from Position 0

consumed: 1

produced: 2 at Position 1

consumed: 2 from Position 1

:

(b) Dining-Philosopher's Problem

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define N 5
sem_t chopstick[N];
void* philosopher(void* num)
{
    int id = *(int*)num;
    while (1)
    {
        printf("Philosopher %d is thinking\n", id);
        sleep(1);
        sem_wait(&chopstick[id]);
        printf("Philosopher %d picked LEFT chopstick %d\n", id, id);
        sem_wait(&chopstick[(id + 1) % N]);
        printf("Philosopher %d picked RIGHT chopstick %d\n",
            id, (id + 1) % N);
        printf("Philosopher %d is eating\n", id);
        sleep(2);
        sem_post(&chopstick[(id + 1) % N]);
        sem_post(&chopstick[id]);
        printf("Philosopher %d finished eating\n", id);
    }
}
int main()
{
    pthread_t ph[N];
```

```

int id[N];

for (int i = 0; i < N; i++)
{
    sem_init(&chopstick[i], 0, 1);
}

for (int i = 0; i < N; i++)
{
    id[i] = i;
    pthread_create(&ph[i], NULL, philosopher, &id[i]);
}

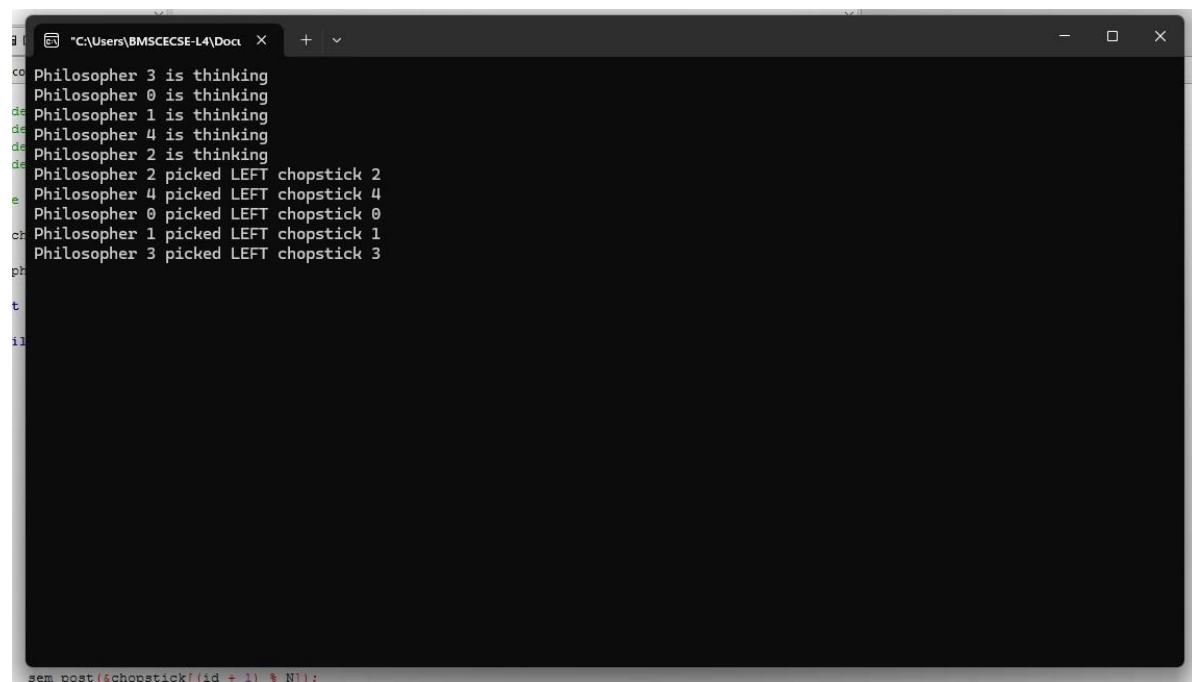
for (int i = 0; i < N; i++)
{
    pthread_join(ph[i], NULL);
}

for (int i = 0; i < N; i++){
    sem_destroy(&chopstick[i]);
}

return 0;
}

```

Output:



```

Philosopher 3 is thinking
Philosopher 0 is thinking
Philosopher 1 is thinking
Philosopher 4 is thinking
Philosopher 2 is thinking
Philosopher 2 picked LEFT chopstick 2
Philosopher 4 picked LEFT chopstick 4
Philosopher 0 picked LEFT chopstick 0
Philosopher 1 picked LEFT chopstick 1
Philosopher 3 picked LEFT chopstick 3

```

Observation:

Program-4

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define N 5

sem_t chopstick[N];
void *philosopher(void *num)
{
    int id = *(int *)num;
    while(1)
    {
        printf("Philosopher %d is thinking\n", id);
        sleep(2);
        sem_wait(&chopstick[id]);
        printf("Philosopher %d picked left chopstick\n", id);
        sem_wait(&chopstick[(id+1)%N]);
        printf("Philosopher %d picked right chopstick\n", id);
    }
}
```

```
int main()
{
    pthread_t p[N];
    int id[N];
    join(int i=0; i<N; i++)
    {
        sem_init(&chopstick[i], 0, 1);
    }
    join(int i=0; i<N; i++)
    {
        sem_init(&chopstick[(i+1)%N], 0, 1);
    }
    join(int i=0; i<N; i++)
    {
        id[i] = i;
        pthread_create(&p[i], NULL, philosopher, &id[i]);
    }
    join(int i=0; i<N; i++)
    {
        pthread_join(p[i], NULL);
    }
    join(int i=0; i<N; i++)
    {
        sem_destroy(&chopstick[i]);
    }
    return 0;
}
```

Date: / /
Page:

Output:

```
Philosopher 1 is thinking
Philosopher 0 is thinking
Philosopher 4 is thinking
Philosopher 3 is thinking
Philosopher 2 is thinking
Philosopher 1 is picked left chopstick 1
Philosopher 2 is picked left chopstick 2
Philosopher 3 is picked left chopstick 3
Philosopher 0 is picked left chopstick 0
```

Lab Program 5:

Write a C program to simulate:

(a) Bankers' algorithm for the purpose of deadlock avoidance

```
#include <stdio.h>
#include <stdbool.h>
#define MAX_PROCESSES 10
#define MAX_RESOURCES 10

int main()
{
    int n, m;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);
    int allocation[MAX_PROCESSES][MAX_RESOURCES];
    int max[MAX_PROCESSES][MAX_RESOURCES];
    int need[MAX_PROCESSES][MAX_RESOURCES];
    int available[MAX_RESOURCES];
    int safeSequence[MAX_PROCESSES];
    bool finish[MAX_PROCESSES];
    printf("\nEnter Allocation Matrix:\n");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            scanf("%d", &allocation[i][j]);
        }
    }
    printf("\nEnter Maximum Matrix:\n");
```

```

for (int i = 0; i < n; i++)
{
    for (int j = 0; j < m; j++)
    {
        scanf("%d", &max[i][j]);
    }
}

printf("\nEnter Available Resources:\n");
for (int i = 0; i < m; i++)
{
    scanf("%d", &available[i]);
}

for (int i = 0; i < n; i++)
{
    for (int j = 0; j < m; j++)
    {
        need[i][j] = max[i][j] - allocation[i][j];
    }
}

for (int i = 0; i < n; i++)
{
    finish[i] = false;
}

int count = 0;
while (count < n)
{
    bool found = false;
    for (int i = 0; i < n; i++)

```



```

{
    if (!finish[i])
    {
        bool possible = true;

        for (int j = 0; j < m; j++)
        {
            if (need[i][j] > available[j])
            {
                possible = false;
                break;
            }
        }
        if (possible)
        {
            for (int j = 0; j < m; j++)
            {
                available[j] += allocation[i][j];
            }
            safeSequence[count] = i;
            count++;
            finish[i] = true;
            found = true;
        }
    }
}

if (!found)
{
    printf("\nSystem is NOT in safe state (Deadlock possible)\n");
    return 0;
}

```

```

    }
}

printf("\nSystem is in SAFE state.\n");
printf("Safe Sequence: ");
for (int i = 0; i < n; i++)
{
    printf("P%d", safeSequence[i]);

    if (i != n - 1)
        printf(" -> ");
}

printf("\n");
return 0;
}

```

Output:

```

"C:\Users\BMSCECSE-L4\Docu... x + - x
Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Maximum Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

System is in SAFE state.
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2

Process returned 0 (0x0)   execution time : 35.002 s
Press any key to continue.

```

Observation:

```

• Banker Algorithm For the purpose of avoidance
Avoidance

#include <stdio.h>
#include <stdlib.h>
#define max_processes 10
#define max_resource 10

int max_req;
int n, m;
printf("Enter number of processes: ");
scanf("%d", &n);
printf("Enter number of resource type: ");
scanf("%d", &m);

int allocation[max_processes][max_resource];
int max_req[max_processes][max_resource];
int need[max_processes][max_resource];

int available[max_resource];
int safe_sequence[max_processes];
bool finish[max_processes];

printf("Enter Allocation matrix: \n");
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < m; j++)
    {
        scanf("%d", &allocation[i][j]);
    }
}

```

```

printf("Enter Maximum matrix: \n");
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < m; j++)
    {
        scanf("%d", &max_req[i][j]);
    }
}

printf("Enter available resources: \n");
for (int i = 0; i < m; i++)
{
    scanf("%d", &available[i]);
}

for (int i = 0; i < n; i++)
{
    for (int j = 0; j < m; j++)
    {
        need[i][j] = max_req[i][j] - allocation[i][j];
    }
}

for (int i = 0; i < n; i++)
{
    finish[i] = false;
}

int count = 0;
while (count < n)
{
    bool found = false;
    for (int i = 0; i < n; i++)
    {
        if (!finish[i] &
            bool possible = true;
            for (int j = 0; j < m; j++)
            {
                if (need[i][j] > available[j])
                {
                    possible = false;
                    break;
                }
            }
            if (possible)
            {
                found = true;
                break;
            }
        }
        if (found)
        {
            safe_sequence[count] = i;
            count++;
            finish[i] = true;
        }
    }
}

if (!found)
{
    printf("System not in safe mode");
    return 0;
}

printf("In system is in safe state: \n");
printf("Safe Sequence: ");
for (int i = 0; i < n; i++)
{
    printf("%d ", safe_sequence[i]);
}
printf("\n");
return 0;
}

```

```

if (need[i][j] > available[j])
{
    possible = false;
    break;
}
}
if (possible)
{
    for (int j = 0; j < m; j++)
    {
        available[j] += allocation[i][j];
    }
}

safe_sequence[count] = i;
count++;
finish[i] = true;
found = true;
}
if (!found)
{
    printf("System not in safe mode");
    return 0;
}

printf("In system is in safe state: \n");
printf("Safe Sequence: ");
for (int i = 0; i < n; i++)
{
    printf("%d ", safe_sequence[i]);
}
printf("\n");
return 0;
}

```

output:-

Enter Number of processes: 5
 Enter Number of resource type: 3
 Enter Allocation matrix:

0	1	0
2	0	0
3	0	2
1	1	2
0	0	2

Enter Maximum Matrix:

7	5	3
3	2	2
9	0	2
2	2	2
4	3	3

Enter Available Resources:

8	3	2
---	---	---

system is in the safe mode
 safe sequence: P1 → P3 → P4 → P2 → P0

(b) Deadlock Detection

```
#include <stdio.h>

#include <stdbool.h>

#define MAX_PROCESSES 10
#define MAX_RESOURCES 10

int main()
{
    int n, m;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resource types: ");
    scanf("%d", &m);

    int allocation[MAX_PROCESSES][MAX_RESOURCES];
    int request[MAX_PROCESSES][MAX_RESOURCES];
    int available[MAX_RESOURCES];
    bool finish[MAX_PROCESSES];
    int safeSequence[MAX_PROCESSES];

    printf("\nEnter Allocation Matrix:\n");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            scanf("%d", &allocation[i][j]);
        }
    }

    printf("\nEnter Request Matrix:\n");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
```

```

        scanf("%d", &request[i][j]);
    }
}
printf("\nEnter Available Resources:\n");
for (int i = 0; i < m; i++)
{
    scanf("%d", &available[i]);
}
for (int i = 0; i < n; i++)
{
    finish[i] = false;
}
int count = 0;
while (count < n)
{
    bool found = false;
    for (int i = 0; i < n; i++)
    {
        if (!finish[i])
        {
            bool possible = true;
            for (int j = 0; j < m; j++)
            {
                if (request[i][j] > available[j])
                {
                    possible = false;
                    break;
                }
            }
        }
    }
}

```

```

        if (possible)
        {
            for (int j = 0; j < m; j++)
            {
                available[j] += allocation[i][j];
            }
            safeSequence[count] = i;
            count++;
            finish[i] = true;
            found = true;
        }
    }
}

if (!found)
{
    break;
}

if (count == n)
{
    printf("\nNo Deadlock Detected.\n");
    printf("Safe Sequence: ");
    for (int i = 0; i < n; i++)
    {
        printf("P%d", safeSequence[i]);
        if (i != n - 1)
            printf(" -> ");
    }
    printf("\n");
}

```

```

else
{
    printf("\nDeadlock Detected!\n");
    printf("Processes in deadlock: ");
    for (int i = 0; i < n; i++)
    {
        if (!finish[i])
        {
            printf("P%d ", i);
        }
    }
    printf("\n");
}
return 0;
}

```

Output:

```

C:\Users\BMSCECSE-L4\Docu...
Enter number of processes: 3
Enter number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 3

Enter Request Matrix:
0 0 0
2 0 0
0 0 1

Enter Available Resources:
0 0 0

Deadlock Detected!
Processes in deadlock: P1 P2

Process returned 0 (0x0)   execution time : 29.729 s
Press any key to continue.

```

Observation:

```

Deadlock Detection:-
#include <stdio.h>
#include <stdbool.h>
#define mp 10
#define mn 10

int main()
{
    int n, m;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);
    int allocation[m][m];
    int request[m][m];
    int available[m];
    bool finish[m];
    int safeSequence[m];

    printf("Enter Allocation Matrix:\n");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            scanf("%d", &allocation[i][j]);
        }
    }

    printf("Enter Request Matrix:\n");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &request[i][0]);
    }
}

```

```

for (int i = 0; i < n; i++)
{
    finish[i] = false;
    while (count == 0)
    {
        bool found = false;
        for (int i = 0; i < n; i++)
        {
            if (!finish[i])
            {
                bool possible = true;
                for (int j = 0; j < m; j++)
                {
                    if (request[i][j] > available[j])
                    {
                        possible = false;
                        break;
                    }
                }
                if (possible)
                {
                    for (int j = 0; j < m; j++)
                    {
                        available[j] += allocation[i][j];
                    }
                    safeSequence[count++] = i;
                    count++;
                    finish[i] = true;
                    found = true;
                }
                if (!found) break;
            }
        }
    }
}

```

```

if (count == n)
{
    printf("No deadlock exists in this system.\n");
    printf("Safe Sequence: ");
    for (int i = 0; i < n; i++)
    {
        printf("%d ", safeSequence[i]);
    }
    if (i == n - 1)
        printf("\n");
}
else
{
    printf("Deadlock detected in\n");
    printf("Processes in Deadlock: ");
    for (int i = 0; i < n; i++)
    {
        if (!finish[i])
        {
            printf("%d ", i);
        }
    }
    printf("\n");
}
return 0;
}

```

```

O/E:-
Enter number of processes: 3
Enter number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
2 0 3

Enter Request Matrix:
0 0 0
2 0 0
0 0 1

Enter Available Resources:
0 0 0

Deadlock detected!
Processes in deadlock: P1 P2

```


Lab Program 6:

Write a C program to simulate the following contiguous memory allocation techniques.

(a) Worst-fit

(b) Best fit

(c) First fit

```
#include <stdio.h>
```

```
void firstFit(int blockSize[], int m, int processSize[], int n)
```

```
{
    int allocation[n];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            if (blockSize[j] >= processSize[i])
            {
                allocation[i] = j;
                blockSize[j] -= processSize[i];
                break;
            }
        }
    }

    printf("\nFirst Fit Allocation:\n");

    printf("Process No.\tProcess Size\tBlock No.\n");

    for (int i = 0; i < n; i++)
    {
        printf("%d\t%d\t", i + 1, processSize[i]);
    }
}
```

```

        if (allocation[i] != -1)

            printf("%d\n", allocation[i] + 1);

        else

            printf("Not Allocated\n");

    }

}

void bestFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[n];

    for (int i = 0; i < n; i++)

        allocation[i] = -1;

    for (int i = 0; i < n; i++)
    {
        int bestIdx = -1;

        for (int j = 0; j < m; j++)
        {
            if (blockSize[j] >= processSize[i])
            {
                if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx])

                    bestIdx = j;
            }
        }

        if (bestIdx != -1)
        {
            allocation[i] = bestIdx;

            blockSize[bestIdx] -= processSize[i];
        }
    }
}

```

```

    }

    printf("\nBest Fit Allocation:\n");

    printf("Process No.\tProcess Size\tBlock No.\n");

    for (int i = 0; i < n; i++)
    {
        printf("%d\t%d\t", i + 1, processSize[i]);

        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void worstFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[n];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;

    for (int i = 0; i < n; i++)
    {
        int worstIdx = -1;

        for (int j = 0; j < m; j++)
        {
            if (blockSize[j] >= processSize[i])
            {
                if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx])
                    worstIdx = j;
            }
        }
    }
}

```

```

    }
    if (worstIdx != -1)
    {
        allocation[i] = worstIdx;
        blockSize[worstIdx] -= processSize[i];
    }
}

printf("\nWorst Fit Allocation:\n");
printf("Process No.\tProcess Size\tBlock No.\n");
for (int i = 0; i < n; i++)
{
    printf("%d\t%d\t", i + 1, processSize[i]);
    if (allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

int main()
{
    int m, n;
    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    int blockSize[m], block1[m], block2[m], block3[m];
    printf("Enter sizes of memory blocks:\n");
    for (int i = 0; i < m; i++)
    {

```

```

scanf("%d", &blockSize[i]);

block1[i] = block2[i] = block3[i] = blockSize[i];
}

printf("Enter number of processes: ");

scanf("%d", &n);

int processSize[n];

printf("Enter sizes of processes:\n");

for (int i = 0; i < n; i++)

    scanf("%d", &processSize[i]);

firstFit(block1, m, processSize, n);

bestFit(block2, m, processSize, n);

worstFit(block3, m, processSize, n);

return 0;

}

```

Output :

```

C:\Users\BMSCECSE-L4\Desktop X + v
Enter number of memory blocks: 5
Enter sizes of memory blocks:
100
500
200
300
600
Enter number of processes: 4
Enter sizes of processes:
212 417 112 426

First Fit Allocation:
Process No.    Process Size    Block No.
1              212            2
2              417            5
3              112            2
4              426            Not Allocated

Best Fit Allocation:
Process No.    Process Size    Block No.
1              212            4
2              417            2
3              112            3
4              426            5

Worst Fit Allocation:
Process No.    Process Size    Block No.
1              212            5
2              417            2
3              112            5
4              426            Not Allocated

Process returned 0 (0x0)   execution time : 22.668 s
Press any key to continue.

```

Observation:

[illegible]

```

void heapFit(int blocks[], int m, int
processes[], int n)
{
    int allocation[10];
    for(int i=0; i<n; i++)
        allocation[i] = -1;
    for(int i=0; i<n; i++)
    {
        int beatIdx = -1;
        for(int j=0; j<m; j++)
        {
            if (blocks[j] >= processes[i])
            {
                if (beatIdx == -1 || blocks[j] < blocks[beatIdx])
                    beatIdx = j;
            }
        }
        if (beatIdx != -1)
        {
            allocation[i] = beatIdx;
            blocks[beatIdx] -= processes[i];
        }
        printf("Heap Fit Allocation: in %d", i);
        printf("Process No %d Process size %d Block No in %d",
            i, processes[i], allocation[i]);
        for(int j=0; j<n; j++)
        {
            printf("%d\t", allocation[j]);
        }
        printf("\n");
        if (allocation[i] == -1)
            printf("Not Allocated in %d", i);
    }
}

```

```

void allocate(int *arr, int n) {
    int i;
    for (i = 0; i < n; i++)
        arr[i] = 0;
}

int main() {
    int arr[10];
    allocate(arr, 10);
    return 0;
}

```

Time	Time	Time
1	100	100
2	100	100
3	100	100
4	100	100
5	100	100
6	100	100
7	100	100
8	100	100
9	100	100
10	100	100
11	100	100
12	100	100
13	100	100
14	100	100
15	100	100
16	100	100
17	100	100
18	100	100
19	100	100
20	100	100
21	100	100
22	100	100
23	100	100
24	100	100
25	100	100
26	100	100
27	100	100
28	100	100
29	100	100
30	100	100
31	100	100
32	100	100
33	100	100
34	100	100
35	100	100
36	100	100
37	100	100
38	100	100
39	100	100
40	100	100
41	100	100
42	100	100
43	100	100
44	100	100
45	100	100
46	100	100
47	100	100
48	100	100
49	100	100
50	100	100
51	100	100
52	100	100
53	100	100
54	100	100
55	100	100
56	100	100
57	100	100
58	100	100
59	100	100
60	100	100
61	100	100
62	100	100
63	100	100
64	100	100
65	100	100
66	100	100
67	100	100
68	100	100
69	100	100
70	100	100
71	100	100
72	100	100
73	100	100
74	100	100
75	100	100
76	100	100
77	100	100
78	100	100
79	100	100
80	100	100
81	100	100
82	100	100
83	100	100
84	100	100
85	100	100
86	100	100
87	100	100
88	100	100
89	100	100
90	100	100
91	100	100
92	100	100
93	100	100
94	100	100
95	100	100
96	100	100
97	100	100
98	100	100
99	100	100
100	100	100

```

int main()
{
    int m, n;
    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    int i, k;
    printf("Enter size of memory block: ");
    for (i = 0; i < m; i++)
    {
        scanf("%d", &blocksize[i]);
        blocksize[i] = blocksize[i] * 1024;
    }
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int processsize[n];
    printf("Enter size of process: ");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &processsize[i]);
    }
    printf("Block, m, process size, n");
    printf("Block, m, process size, n");
    printf("Block, m, process size, n");
    return 0;
}

```

Q:-
Enter number of memory blocks: 3
Enter size of memory blocks:
10 12 150
Enter number of processes: 3
Enter size of process: 100 10 100

Lab Program 7

Write a C program to simulate page replacement algorithms.

a) FIFO

b) LRU

c) Optimal

```
#include <stdio.h>

#define MAX 50

void fifo(int pages[], int n, int frames)
{
    int frame[MAX], i, j, k = 0, fault = 0, found;
    for(i = 0; i < frames; i++)
        frame[i] = -1;
    printf("\nFIFO Page Replacement:\n");
    for(i = 0; i < n; i++)
    {
        found = 0;
        for(j = 0; j < frames; j++)
        {
            if(frame[j] == pages[i])
            {
                found = 1;
                break;
            }
        }
        if(!found)
        {
            frame[k] = pages[i];
            k = (k + 1) % frames;
            fault++;
        }
    }
}
```

```

    }

    printf("Page %d -> ", pages[i]);
    for(j = 0; j < frames; j++)
        printf("%d ", frame[j]);
    printf("\n");
}

printf("Total Page Faults = %d\n", fault);
}

void lru(int pages[], int n, int frames)
{
    int frame[MAX], time[MAX];
    int i, j, pos, fault = 0, counter = 0, found;

    for(i = 0; i < frames; i++)
    {
        frame[i] = -1;
        time[i] = 0;
    }

    printf("\nLRU Page Replacement:\n");

    for(i = 0; i < n; i++)
    {
        found = 0;
        for(j = 0; j < frames; j++)
        {
            if(frame[j] == pages[i])
            {
                counter++;
                time[j] = counter;
                found = 1;
            }
        }
    }
}

```



```

        break;
    }
}
if(!found)
{
    pos = 0;
    for(j = 1; j < frames; j++)
    {
        if(time[j] < time[pos])
            pos = j;
    }
    frame[pos] = pages[i];
    counter++;
    time[pos] = counter;
    fault++;
}
printf("Page %d -> ", pages[i]);
for(j = 0; j < frames; j++)
    printf("%d ", frame[j]);
printf("\n");
}
printf("Total Page Faults = %d\n", fault);
}
void optimal(int pages[], int n, int frames)
{
    int frame[MAX];
    int i, j, k, pos, fault = 0, found;

    for(i = 0; i < frames; i++)
        frame[i] = -1;

```

```
printf("\nOptimal Page Replacement:\n");
```

```
for(i = 0; i < n; i++)  
{  
    found = 0;  
    for(j = 0; j < frames; j++)  
    {  
        if(frame[j] == pages[i])  
        {  
            found = 1;  
            break;  
        }  
    }  
    if(!found)  
    {  
        int farthest = -1;  
        for(j = 0; j < frames; j++)  
        {  
            int nextUse = 9999;  
            for(k = i + 1; k < n; k++)  
            {  
                if(frame[j] == pages[k])  
                {  
                    nextUse = k;  
                    break;  
                }  
            }  
            if(nextUse > farthest)  
            {  
                farthest = nextUse;  
            }  
        }  
    }  
}
```

```

        pos = j;
    }
}

    frame[pos] = pages[i];
    fault++;
}
printf("Page %d -> ", pages[i]);
for(j = 0; j < frames; j++)
    printf("%d ", frame[j]);
printf("\n");
}

printf("Total Page Faults = %d\n", fault);
}

int main()
{
    int n, frames, i;
    int pages[MAX];

    printf("Enter number of pages: ");
    scanf("%d", &n);

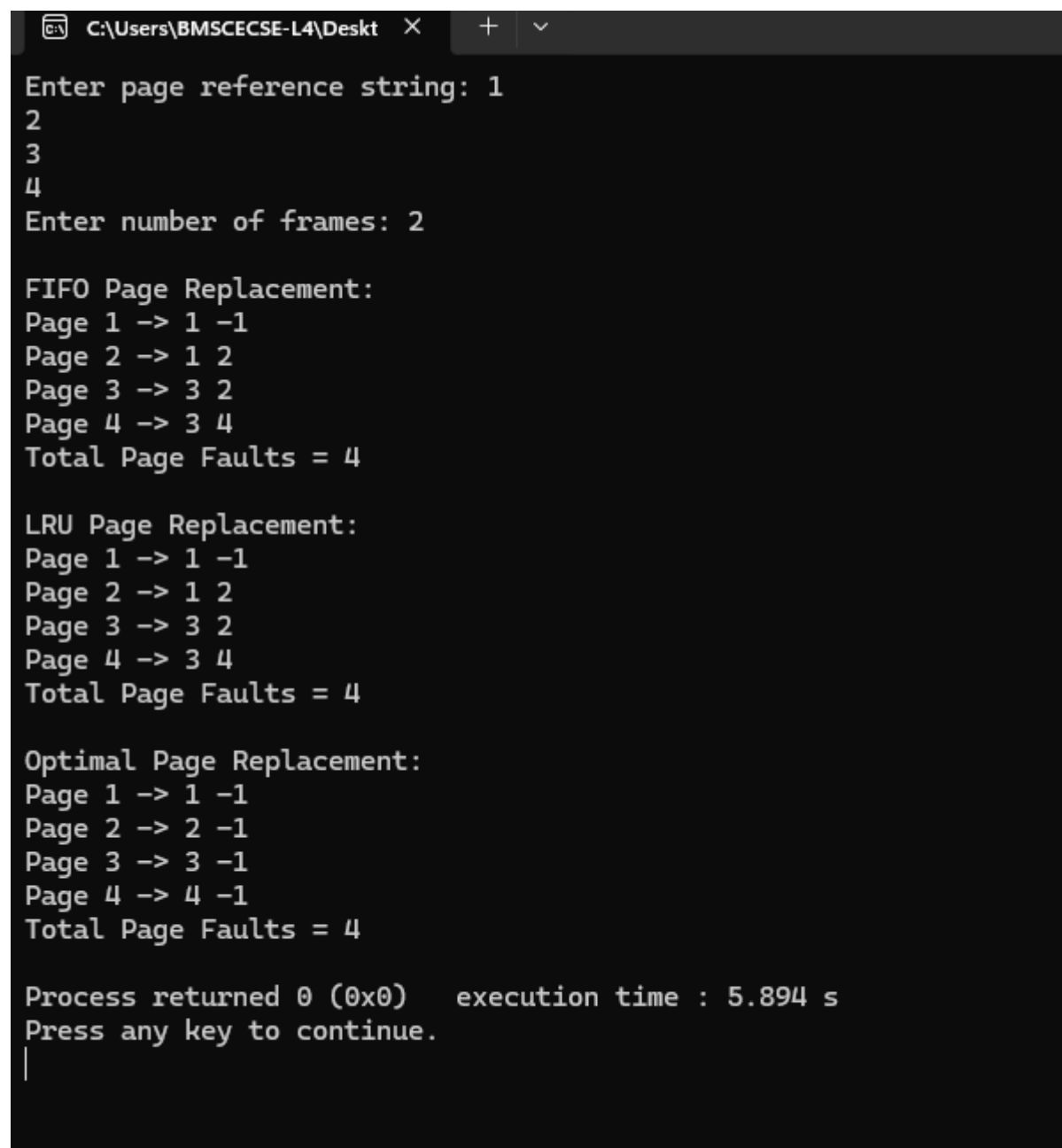
    printf("Enter page reference string: ");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &frames);

```

```
fifo(pages, n, frames);  
lru(pages, n, frames);  
optimal(pages, n, frames);  
  
return 0;  
}
```

Output :



```
C:\Users\BMSCECSE-L4\Desktop X + v  
Enter page reference string: 1  
2  
3  
4  
Enter number of frames: 2  
  
FIFO Page Replacement:  
Page 1 -> 1 -1  
Page 2 -> 1 2  
Page 3 -> 3 2  
Page 4 -> 3 4  
Total Page Faults = 4  
  
LRU Page Replacement:  
Page 1 -> 1 -1  
Page 2 -> 1 2  
Page 3 -> 3 2  
Page 4 -> 3 4  
Total Page Faults = 4  
  
Optimal Page Replacement:  
Page 1 -> 1 -1  
Page 2 -> 2 -1  
Page 3 -> 3 -1  
Page 4 -> 4 -1  
Total Page Faults = 4  
  
Process returned 0 (0x0) execution time : 5.894 s  
Press any key to continue.  
|
```

